



Departamento de Computación y Sistemas Inteligentes

Ingeniería de Sistemas

Seminario de Ingeniería de Software

Laboratorio desafío: App web desplegada en la nube con servicios open source y administrados

Profesores: Gonzalo Llano R Ph.D /José Luis Jurado Ph. D

Grupo No.: 06

Integrantes del grupo: Camilo Toro Agudelo; Maria Alejandra Muñoz Ruiz; Heidy Yuliana Hernández Huila; Sarah Rodriguez Ardila; María Camila Salamanca Cortés

Santiago de Cali, septiembre del 2026

Informe técnico final

Laboratorio desafío: CloudTasks — App web desplegada en la nube con servicios open source y administrados

1) Introducción

En este laboratorio se desarrollará **CloudTasks**, una aplicación web para la gestión de tareas personales o de un equipo de trabajo. El propósito no es únicamente construir una aplicación funcional, sino comprender cómo una solución de software puede evolucionar progresivamente desde un entorno local hasta una aplicación disponible en Internet utilizando servicios open source administrados de computación en la nube.

El laboratorio se desarrollará mediante **tres etapas progresivas**: **Etapla 1:** Se debe diseñar y desarrollar la app web utilizando **HTML, CSS y JavaScript**, aplicando buenas prácticas básicas de desarrollo y controlando la evolución del código mediante **Git**, para posteriormente almacenarlo y gestionarlo en **GitHub**.

Etapla 2: la aplicación debe ser desplegada en la nube mediante el uso de los servicios *open source* **Vercel** para construir y desplegar el **frontend**, y **Supabase** para el **Backend**, y de **persistencia de datos** con PostgreSQL. De esta manera, los estudiantes podrán observar la transición de una aplicación local hacia una solución en la nube que utiliza servicios administrados o *serverless*.

Etapla 3: Se debe incorporar en el pipeline el servicio *open source* de **Cloudflare** al flujo de **acceso seguro de la aplicación**. Se debe configurar el dominio, DNS, HTTPS y el acceso hacia la aplicación desplegada en Vercel, comprendiendo el papel que desempeña cada servicio dentro del flujo completo de la solución.

El laboratorio busca que el estudiante comprenda que en diseñar y desplegar una aplicación web en la nube, no consiste simplemente en "subir una aplicación a Internet", sino en **integrar diferentes servicios especializados para construir una solución funcional, accesible, escalable y gestionable**.

2) Objetivo general

Diseñar, desarrollar y desplegar una aplicación web funcional utilizando HTML, CSS y JavaScript, integrando Git, GitHub, Supabase, Vercel y Cloudflare para comprender progresivamente el ciclo de desarrollo, control de versiones, persistencia de datos, despliegue y acceso seguro a una aplicación en la nube.

3) Requerimientos funcionales de la aplicación CloudTaks

A partir del enunciado del laboratorio y de la versión desarrollada de CloudTasks, se definieron los siguientes requerimientos funcionales. Los seis primeros responden directamente a la gestión básica de tareas exigida y los restantes corresponden a funcionalidades incorporadas por el equipo para organizar el trabajo por roles.

Requerimientos funcionales

ID	Requerimiento	Descripción
RF-01	Crear tareas	El sistema debe permitir registrar una tarea con título, descripción, fecha límite, prioridad y estado.

ID	Requerimiento	Descripción
RF-02	Visualizar tareas	El usuario debe poder consultar las tareas disponibles de acuerdo con su sesión y rol.
RF-03	Actualizar estado	Debe ser posible marcar una tarea como completada o modificar su estado cuando el rol tenga permiso.
RF-04	Eliminar tareas	Las tareas deben poder eliminarse desde la aplicación cuando el usuario autorizado lo solicite.
RF-05	Mostrar estado	Cada tarea debe presentar su estado, prioridad y fecha límite de forma visible.
RF-06	Validar datos	Los campos obligatorios deben validarse antes de crear o modificar una tarea.
RF-07	Autenticación y roles	La interfaz contempla inicio de sesión y perfiles administrador, líder y usuario para controlar visibilidad y acciones.
RF-08	Búsqueda y filtros	La aplicación permite buscar tareas y filtrar por prioridad para facilitar su consulta.
RF-09	Asignación de responsables	Un administrador puede asociar un líder y usuarios a una tarea.
RF-10	Persistencia cloud	La arquitectura contempla almacenar la información mediante Supabase sobre PostgreSQL.

Requerimientos no funcionales

ID	Atributo	Criterio
RNF-01	Usabilidad	La interfaz debe ser clara, consistente y comprensible para los distintos roles.
RNF-02	Seguridad básica	No se deben publicar contraseñas ni claves privadas; el frontend solo puede utilizar credenciales públicas apropiadas.
RNF-03	Trazabilidad	El código debe mantenerse en Git/GitHub para conservar historial de cambios y facilitar el trabajo colaborativo.

4) Diseño de la app y tecnologías utilizadas

CloudTasks se diseñó como una aplicación web del lado del cliente construida con HTML, CSS y JavaScript. La arquitectura objetivo separa el control de versiones, el despliegue del frontend, el acceso por dominio y la persistencia de datos en servicios especializados. Esto permite explicar con claridad qué responsabilidad tiene cada componente dentro del flujo completo.

Arquitectura de alto nivel

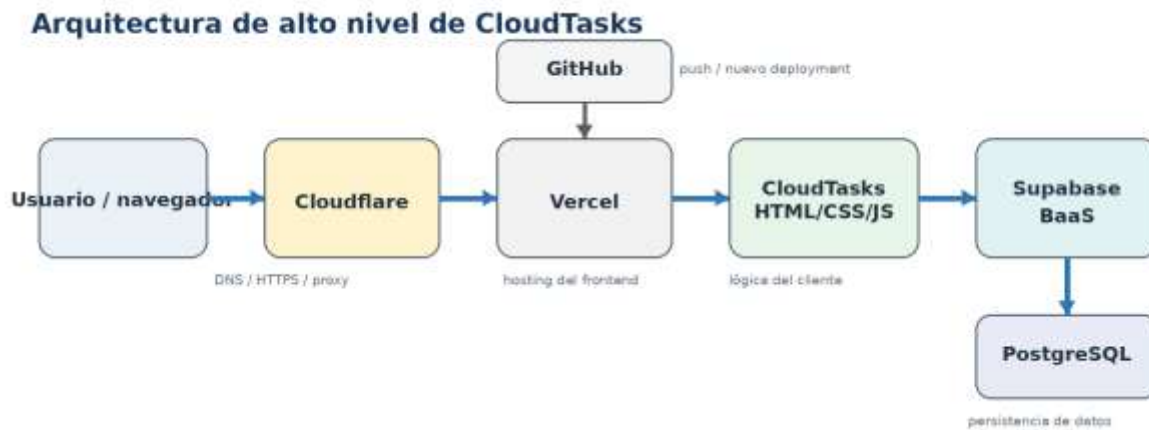


Figura 1. Arquitectura objetivo de alto nivel de CloudTasks.

Tecnologías utilizadas

Tecnología / servicio	Función en CloudTasks
HTML	Estructura de la interfaz y formularios.
CSS	Presentación visual y distribución de los componentes.
JavaScript	Lógica de interacción, validaciones, tareas, filtros y comunicación con servicios.
Git	Control de versiones local.
GitHub	Repositorio remoto, colaboración e historial del proyecto.
Vercel	Despliegue y publicación del frontend.
Supabase	Backend as a Service para autenticación y acceso a datos.
PostgreSQL	Persistencia de perfiles, tareas y asignaciones.
Cloudflare	Componente previsto para DNS, HTTPS/TLS y acceso mediante dominio en la etapa final.

Flujo de datos propuesto

1. El usuario accede a CloudTasks desde el navegador utilizando el dominio de la aplicación.
2. Cloudflare resuelve el dominio y, en la arquitectura final, encamina el acceso seguro hacia el despliegue.
3. Vercel entrega los archivos del frontend de CloudTasks al navegador.
4. JavaScript ejecuta la lógica de la aplicación y se comunica con Supabase para autenticación y operaciones sobre las tareas.
5. Supabase utiliza PostgreSQL para conservar los datos de forma persistente.
6. En el flujo de despliegue, los cambios integrados en GitHub pueden generar una nueva versión publicada por Vercel.

Estado final de la documentación: el informe consolida los requerimientos, arquitectura, desarrollo local, Git/GitHub, Vercel, Supabase, pruebas, uso de inteligencia artificial y el avance real de Cloudflare. La aplicación está

publicada y operativa mediante Vercel; la activación del dominio personalizado continúa condicionada a la aprobación externa de EU.org.

5) Etapa 1 (desarrollo local de la app)

CloudTasks se construyó inicialmente con HTML, CSS y JavaScript para validar la interfaz y la lógica antes de integrar servicios cloud. La aplicación incorpora autenticación, roles, gestión de tareas, filtros y validaciones.

5.1 Funcionalidades y estructura local

La solución separa la interfaz (index.html), los estilos (css/styles.css), la lógica principal (js/app.js), la configuración de Supabase (js/supabase-config.js), las pruebas y la documentación. La aplicación maneja los roles administrador, líder y usuario, con permisos distintos sobre las tareas.

Función	Comportamiento principal
Gestión de tareas	Crear, visualizar, editar, completar y eliminar según permisos.
Organización	Estados, prioridad, fecha límite, búsqueda y filtros.
Roles	Administrador gestiona; líder actualiza sus tareas; usuario consulta las asignadas.
Validaciones	Control de campos obligatorios, fechas, estados y contenido mostrado.

5.2 Pruebas de la etapa local

El repositorio incluye pruebas unitarias aisladas del DOM y de servicios externos. Se ejecutaron 51 casos sobre funciones de normalización, fechas, visibilidad, ordenamiento, escape de HTML y etiquetas de interfaz; los 51 obtuvieron el resultado esperado.

Tipo	Casos
Normales	32
Error	12
Límite	7
Total aprobado	51/51

6) Etapa 2 (Github)

El proyecto se versiona con Git y se almacena en GitHub. El repositorio permite conservar el historial, integrar cambios entre integrantes y servir como origen de los despliegues posteriores.

6.1 Flujo de trabajo

Durante el desarrollo de CloudTasks se utilizó GitHub como repositorio remoto y se trabajó con varias ramas para separar funcionalidades, pruebas, correcciones e integración. La rama main se mantuvo como rama principal del proyecto, mientras que las demás ramas permitieron desarrollar y validar cambios antes de incorporarlos a la versión estable.

El repositorio cuenta con 7 ramas visibles: main, DBVercel, Design, Users, develop, test y testFix. Esta organización permitió dividir el trabajo por áreas y reducir el riesgo de modificar directamente la rama principal durante el desarrollo.

El flujo general consistió en desarrollar cambios en ramas específicas, verificar su funcionamiento y posteriormente integrarlos al proyecto principal. De esta forma, el equipo pudo trabajar en paralelo y conservar un historial de evolución del sistema. En la evidencia actual del repositorio también se observan 33 commits, lo que permite rastrear los cambios realizados a lo largo del proyecto.

Estrategia de ramas

Rama	Propósito dentro del proyecto
main	Rama principal y versión estable del proyecto.
develop	Rama utilizada para integrar cambios antes de incorporarlos a main.
Design	Rama orientada a cambios de diseño e interfaz de usuario.
Users	Rama destinada a funcionalidades relacionadas con usuarios y administración.
DBVercel	Rama asociada a cambios de base de datos e integración/despliegue con Vercel.
test	Rama utilizada para implementar y ejecutar pruebas.
testFix	Rama destinada a correcciones derivadas de los resultados de las pruebas.

Resumen del flujo aplicado

- Crear o utilizar una rama específica según el tipo de cambio.
- Desarrollar y probar la funcionalidad o corrección en esa rama.
- Registrar los cambios mediante commits descriptivos.
- Enviar los cambios al repositorio remoto con push.
- Integrar los cambios aprobados hacia develop o main según el estado del proyecto.
- Mantener main como referencia de la versión estable para entrega y despliegue.

7) Etapa 3 (Integración con Vercel)

CloudTasks se desplegó en Vercel a partir del repositorio del proyecto, permitiendo acceder a la aplicación desde Internet y probar la integración con Supabase fuera del entorno local.

7.1 Despliegue y validación

La versión publicada está disponible en <https://cloudtasks-equipo06.vercel.app/>. Vercel sirve el frontend y utiliza el repositorio como fuente de la aplicación. Después de los cambios integrados, el equipo verifica que la URL pública cargue correctamente y mantenga las funcionalidades principales.

Prueba	Resultado esperado
Acceso público	La URL carga CloudTasks correctamente.
Recursos	HTML, CSS y JavaScript se cargan sin errores visibles.
Integración	Login y operaciones de tareas funcionan desde la versión desplegada.
Actualización	Los cambios integrados en el repositorio se reflejan en un nuevo despliegue.

8) Etapa 3 (Integración con Supabase)

Supabase funciona como Backend as a Service y PostgreSQL como sistema de persistencia. La aplicación utiliza Supabase Auth, perfiles por rol y operaciones CRUD sobre tareas y asignaciones.

8.1 Modelo e integración

Tabla	Uso
profiles	Información del usuario autenticado y su rol.
tasks	Datos principales de la tarea, estado, prioridad, deadline y líder.
task_assignments	Relación entre tareas y usuarios asignados.

JavaScript crea el cliente de Supabase con la URL pública y la anon key. Después del login se recupera el perfil del usuario y se cargan las tareas correspondientes a su rol. Las operaciones de creación, consulta, actualización y eliminación se realizan contra PostgreSQL mediante el SDK de Supabase.

8.2 CRUD, roles y persistencia

Aspecto	Implementación
Create	El administrador crea tareas y registra sus asignaciones.
Read	Se consultan tareas y asignaciones según el usuario autenticado.
Update	Administrador edita; líder puede actualizar el estado de sus tareas.
Delete	La eliminación está restringida al rol con permisos administrativos.
Persistencia	Los datos se mantienen después de recargar la aplicación.

El README documenta el esquema de profiles, tasks y task_assignments, junto con políticas RLS para controlar la visibilidad y modificación de datos por rol. La integración utiliza Supabase Auth, la anon key pública y el SDK de Supabase desde JavaScript; la clave service_role no se expone en el frontend.

9) Etapa 3 (Integración con Cloudflare)

La integración con Cloudflare fue preparada para incorporar un dominio personalizado, resolución DNS y HTTPS delante del despliegue de Vercel. El equipo creó la cuenta de Cloudflare, seleccionó el plan Free y solicitó el dominio clouttasks.eu.org a través de EU.org.

9.1 Configuración realizada

Elemento	Estado / valor
Aplicación en Vercel	https://clouttasks-equipo06.vercel.app/
Dominio solicitado	clouttasks.eu.org
Plan Cloudflare	Free
Nameserver 1	connie.ns.cloudflare.com

Elemento	Estado / valor
Nameserver 2	otto.ns.cloudflare.com
Solicitud EU.org	20260909222719-arf-34219
Comprobación técnica	Validaciones SOA y NS completadas correctamente
Estado del dominio	Pendiente de aprobación por EU.org

9.2 Flujo previsto y estado actual

Mientras el dominio permanece pendiente, el flujo operativo validado es Usuario → Vercel → CloudTasks → Supabase → PostgreSQL. Una vez EU.org apruebe el dominio, el flujo objetivo será Usuario → Cloudflare → Vercel → CloudTasks → Supabase → PostgreSQL. Cloudflare resolverá el dominio y administrará el acceso HTTPS, mientras Vercel continuará sirviendo el frontend.

9.3 Validación de Cloudflare

Prueba	Resultado actual
Registro del dominio	Solicitud creada en EU.org y asociada a los nameservers de Cloudflare.
Nameservers	Comprobaciones NS y SOA superadas.
DNS hacia Vercel	Pendiente hasta que EU.org delegue el dominio.
HTTPS/TLS del dominio	Pendiente de la activación DNS.
Acceso público	Disponible actualmente mediante la URL de Vercel.

La demora corresponde a una dependencia externa de aprobación del dominio. El archivo doc/cloudflare.md conserva el detalle del trámite y permite continuar la configuración cuando la delegación sea aprobada.

9.4 Arquitectura y flujo de despliegue

El código se versiona en GitHub y la versión publicada se despliega en Vercel. El navegador ejecuta el frontend y JavaScript se comunica con Supabase para autenticación y persistencia. Cloudflare se incorpora como capa de DNS y acceso seguro cuando el dominio quede activo. De este modo, cada servicio conserva una responsabilidad separada dentro de la arquitectura.

10) Uso de inteligencia artificial

Durante el laboratorio se utilizó ChatGPT como herramienta de apoyo para planificación, revisión técnica, documentación y resolución de dudas puntuales. Las respuestas de IA se trataron como propuestas que debían ser revisadas y adaptadas por el equipo antes de incorporarlas al proyecto.

El equipo revisó las soluciones incorporadas y deberá poder explicar durante la sustentación el código, las pruebas y las decisiones técnicas, independientemente del apoyo recibido por herramientas de IA.

10.1 Registro de uso de IA

Aspecto	Aplicación en el proyecto
Herramienta	ChatGPT (OpenAI).

Aspecto	Aplicación en el proyecto
Propósito	Organizar el trabajo, resolver dudas de Git, revisar el estado del proyecto, apoyar la documentación y detectar riesgos de integración.
Prompts representativos	Lee el documento del laboratorio CloudTasks y explícame paso a paso qué pide el proyecto, cómo debemos iniciarlo y cómo podemos repartir el trabajo entre cinco integrantes Revisa el estado actual del proyecto y dime qué funcionalidades o integraciones importantes faltan antes de la entrega
Soluciones obtenidas	Sugerencias de flujo de ramas, comandos Git, revisión de integración, estructura del informe y criterios para documentar pruebas y servicios cloud.
Cambios del equipo	Las propuestas se ajustaron a la estructura real del repositorio, roles, nombres de archivos, esquema de Supabase y estado real de Cloudflare.
Validación	Se contrastaron las propuestas con el código, README, documentación del repositorio y ejecución de pruebas. La suite actual contiene 51 casos y 51 fueron aprobados.
Errores / ajustes	Algunas recomendaciones iniciales correspondían a versiones anteriores del proyecto y tuvieron que ser revisadas al actualizar el repositorio. No se asumieron como válidas sin comprobarlas.
Limitaciones	La IA no puede sustituir pruebas reales, credenciales, aprobaciones de terceros ni la comprensión del código. La aprobación de EU.org, por ejemplo, depende de un servicio externo y no puede resolverse mediante IA.

10.2 Problemas encontrados y soluciones

Problema	Solución aplicada
Persistencia de tareas	Se migró del modo local a Supabase/PostgreSQL y se integró el SDK desde JavaScript.
Permisos por rol	Se implementaron perfiles, visibilidad diferenciada y políticas RLS documentadas para reforzar el control desde la base de datos.
Estados y vencimientos	Se centralizó la normalización de estados y se agregaron casos unitarios para fechas, prioridades y tareas vencidas.
Pruebas sin DOM	Las funciones puras se aislaron en tests/funciones.js y se creó tests/runner.html para ejecutarlas de forma repetible.
Integración de cambios	Se utilizó Git con main/develop y se revisaron los cambios antes de integrarlos; ante merges problemáticos se contempló git merge --abort.
Dominio Cloudflare	Se configuraron nameservers y se validó SOA/NS; la finalización queda pendiente de la aprobación externa de EU.org.

10.3 Evidencias del proyecto

- Repositorio: <https://github.com/MissyMilaS/cloudtasks-equipo06.git>
- Aplicación publicada: <https://cloudtasks-equipo06.vercel.app/>
- README.md: arquitectura, roles, esquema SQL, RLS, despliegue y equipo.
- doc/plan_de_pruebas.md y tests/runner.html: 51 casos de prueba; ejecución verificada con 51/51 aprobados.
- doc/cloudflare.md: dominio solicitado, nameservers, identificador de solicitud y estado del trámite.

- `js/app.js` y `js/supabase-config.js`: integración de frontend, Supabase Auth, CRUD y configuración pública del cliente.

10.4 Conclusiones

CloudTasks evolucionó desde una aplicación local en HTML, CSS y JavaScript hasta una solución publicada en Vercel y conectada a Supabase/PostgreSQL para autenticación y persistencia. El proyecto incorporó control de versiones, roles, políticas de acceso, CRUD y una suite de pruebas que permite verificar la lógica principal de manera repetible.

La integración de servicios permitió separar responsabilidades: GitHub conserva la evolución del código, Vercel publica el frontend, Supabase administra autenticación y datos, y Cloudflare queda preparado como capa de DNS y HTTPS. La única dependencia pendiente al cierre de este informe es la aprobación de `cloudtasks.eu.org` por EU.org; una vez delegado el dominio, podrán completarse DNS, asociación con Vercel y validación HTTPS sin modificar la arquitectura base.

11) Criterio general de evaluación

La evaluación tendrá en cuenta el funcionamiento de la app web, así como, el proceso de ingeniería utilizado para construirlo. Se valorará especialmente:

- Funcionalidad de CloudTasks.
- Calidad del código HTML/CSS/JavaScript.
- Uso adecuado de Git y GitHub.
- Persistencia e integración con Supabase.
- Despliegue correcto en Vercel.
- Configuración de Cloudflare.
- Comprensión de la arquitectura.
- Integración entre los diferentes servicios.
- Calidad de las pruebas.
- Documentación técnica.
- Capacidad para explicar las decisiones tomadas.
- Uso responsable y crítico de herramientas de IA.